

Tests Unitaires et Mocking avec Pester

1. Cadrage Opérationnel

L'objectif est d'écrire une suite automatisée de tests unitaires et de validation d'infrastructure pour éprouver le module **InfraTools** en isolation totale : **zéro paquet envoyé sur le réseau, zéro contact avec le matériel physique et zéro pollution du disque dur.**

Vous utiliserez le framework **Pester** pour :

- Pénétrer le scope privé scellé du module (InModuleScope) pour auditer le moteur interne de journalisation.
- Exploiter le lecteur temporaire virtuel (\$TestDrive) pour tester la création de fichiers et de répertoires sans laisser de traces.
- Simuler les réponses matérielles de Windows (Mock Get-CimInstance).
- Simuler les tests de connectivité réseau sans dépendre de serveurs distants (Mock Test-NetConnection).
- Traquer les effets de bord et vérifier les appels internes (Assert-MockCalled).
- Éprouver la robustesse face aux entrées invalides (Should -Throw) et au flux du pipeline (|).

2. Organisation des Fichiers de Test

Dans votre dossier de travail InfraTools, implémentez l'arborescence de test suivante :

```
InfraTools/  
├── InfraTools.psd1  
├── InfraTools.psm1  
├── Public/  
│   ├── Get-InfraDisque.ps1  
│   └── Test-FluxPort.ps1  
├── Private/  
│   ├── Write-JournalModule.ps1  
│   └── Convert-TailleOctet.ps1  
└── Tests/  
    ├── Write-JournalModule.Tests.ps1  
    ├── Get-InfraDisque.Tests.ps1  
    └── Test-FluxPort.Tests.ps1
```

3. Cahier des Charges des Fichiers de Test

Étape 1 : Tests de la Fonction Interne de Journalisation
(Tests\Write-JournalModule.Tests.ps1)

Cette suite valide la fonction utilitaire privée Write-JournalModule sans modifier son étanchéité dans le module.

1. Conteneur global et accès au scope privé :

- Déclarer un bloc Describe "Write-JournalModule".
- Dans un bloc BeforeAll, importer le module InfraTools (-Force).
- Encapsuler l'ensemble des scénarios dans l'instruction de pénétration de session (InModuleScope "InfraTools").

2. Scénario 1 : Écriture sécurisée et formatage horodaté dans \$TestDrive :

- Définir un Context "Formatage et écriture du journal".
- Rédiger un test unitaire (It) qui appelle Write-JournalModule avec un message et le niveau "INFO", en redirigeant temporairement le chemin cible vers le disque virtuel \$TestDrive:\infratools.log.
- Valider que le fichier est créé sur \$TestDrive (Test-Path | Should -BeTrue).
- Lire le contenu du fichier et valider par expression régulière (Should -Match) le respect du masque strict :
`^[\\d{4}-\\d{2}-\\d{2} \\d{2}:\\d{2}:\\d{2}] \\[INFO\\] Mon message`

3. Scénario 2 : Création dynamique du répertoire parent :

- Définir un Context "Résilience de l'arborescence".
- Rédiger un test ciblant un sous-dossier inexistant (ex :
\$TestDrive:\SousDossierAudit\journal.log).
- Exécuter Write-JournalModule.
- Valider que le sous-dossier et le fichier ont été créés sans erreur.

4. Scénario 3 : Rejet des niveaux de gravité interdits :

- Définir un Context "Contrôle des paramètres".
- Rédiger un test vérifiant qu'un appel avec un niveau hors catalogue (ex : -Niveau "DEBUG" ou "FATAL") est immédiatement intercepté et bloqué ({ ... } | Should -Throw).

Étape 2 : Tests d'Isolation Matérielle (Tests\Get-InfraDisque.Tests.ps1)

Cette suite teste la logique décisionnelle de Get-InfraDisque sans jamais solliciter les contrôleurs de disques physiques ni écrire de logs parasites.

1. Conteneur et préparation :

- Déclarer un bloc Describe "Get-InfraDisque".
- Dans un bloc BeforeAll, charger le module InfraTools.
- Dans un bloc BeforeEach, neutraliser tout appel à Write-JournalModule via un Mock vierge.

2. Scénario 1 : Partition saine avec espace disponible abondant :

- Définir un Context "Lorsque la partition dispose d'un espace supérieur au seuil".
- Configurer un Mock sur Get-CimInstance filtré sur la classe Win32_LogicalDisk simulant un disque de 100Go avec 60Go libre (60% libre).
- Appeler Get-InfraDisque -Lecteur "C".
- Valider que le retour est bien un objet [PSCustomObject].
- Valider que la propriété Statut vaut strictement "OPTIMAL".

- Valider que la propriété LibrePct vaut 60.0.
 - Prouver avec Assert-MockCalled que Write-JournalModule n'a **jamais** été appelée.
3. **Scénario 2 : Partition saturée franchissant le seuil d'alerte :**
- Définir un Context "Lorsque la partition franchit le seuil critique à la baisse".
 - Configurer un Mock sur Get-CimInstance simulant un disque de 100Go avec seulement 8Go d'espace libre (8% libre).
 - Appeler Get-InfraDisque -Lecteur "C" -SeuilAlertePct 15.
 - Valider que la propriété Statut vaut "CRITIQUE".
 - Valider que la propriété LibrePct vaut 8.0.
 - Prouver avec Assert-MockCalled que Write-JournalModule a été appelée **exactement 1 fois** avec le niveau "WARNING".
4. **Scénario 3 : Prise en charge du pipeline :**
- Définir un Context "Transmission multi-cibles par pipeline".
 - Configurer un Mock sur Get-CimInstance retournant un disque factice valide quel que soit le lecteur.
 - Transmettre deux lettres de lecteurs via le pipeline : @("C", "D") | Get-InfraDisque.
 - Valider que la collection de sortie comporte exactement deux enregistrements (Should -HaveCount 2).

Étape 3 : Tests de Connectivité Réseau Simulée (Tests\Test-FluxPort.Tests.ps1)

Cette suite valide le comportement de Test-FluxPort sans envoyer la moindre trame sur le réseau physique de votre machine.

1. **Conteneur et préparation :**
- Déclarer un bloc Describe "Test-FluxPort".
 - Dans un bloc BeforeAll, importer le module InfraTools.
 - Dans un bloc BeforeEach, neutraliser la fonction Write-JournalModule avec un Mock vierge.
2. **Scénario 1 : Port réseau distant ouvert et accessible :**
- Définir un Context "Lorsque le port distant est accessible".
 - Configurer un Mock sur la commande native réseau (Test-NetConnection) simulant une connexion réussie (TcpTestSucceeded = \$true).
 - Appeler Test-FluxPort -Cibles "192.168.1.10" -Port 443.
 - Valider que la propriété Ouvert de l'objet de sortie vaut strictement \$true.
 - Valider que la propriété Cible correspond bien à "192.168.1.10".
 - Valider que la propriété DateControle n'est ni nulle ni vide (Should -Not -BeNullOrEmpty).
3. **Scénario 2 : Port fermé ou bloqué par pare-feu :**
- Définir un Context "Lorsque le port distant est fermé ou injoignable".
 - Configurer un Mock sur Test-NetConnection simulant un échec de connexion (TcpTestSucceeded = \$false).
 - Appeler Test-FluxPort -Cibles "10.0.0.1" -Port 8080.

- Valider que la propriété Ouvert vaut strictement \$false.
 - Valider avec Assert-MockCalled que l'alerte a été consignée dans le journal via Write-JournalModule.
4. **Scénario 3 : Rejet strict des numéros de port invalides :**
- Définir un Context "Validation des bornes de ports TCP".
 - Rédiger un test vérifiant qu'un appel avec un port inférieur à 1 (ex : -Port 0) est rejeté net (Should -Throw).
 - Rédiger un second test vérifiant qu'un appel avec un port supérieur à la limite physique 65 535 (ex : -Port 70000) est rejeté (Should -Throw).
5. **Scénario 4 : Pipeline réseau multi-serveurs :**
- Définir un Context "Traitement de cibles en masse".
 - Configurer un Mock sur Test-NetConnection renvoyant une réponse simulée.
 - Injecter un tableau de 3 adresses IP via le pipeline : @("10.0.0.1", "10.0.0.2", "10.0.0.3") | Test-FluxPort -Port 443.
 - Valider que la collection finale contient exactement 3 objets (Should -HaveCount 3).

4. Protocole d'Exécution et Recette

Exécutez vos tests dans votre console PowerShell et observez les métriques.

Commande 1 : Exécution globale détaillée

Positionnez-vous à la racine du module et lancez la campagne de tests :

```
Invoke-Pester -Path ".\Tests" -Output Detailed
```

- **Résultat obligatoire :**
 - L'ensemble des blocs de tests (Write-JournalModule, Get-InfraDisque, Test-FluxPort) s'affichent avec des coches vertes [+].
 - Le bilan récapitulatif final doit afficher : Failed: 0.

Commande 2 : Audit de Couverture de Code (Code Coverage)

Vérifiez que vos tests parcourent l'ensemble de votre logique métier :

```
$Config = New-PesterConfiguration
$Config.Run.Path = ".\Tests"
$Config.CodeCoverage.Enabled = $true
$Config.CodeCoverage.Path = ".\Public\*.ps1", ".\Private\*.ps1"
```

```
Invoke-Pester -Configuration $Config
```

- **Résultat obligatoire :**
 - Le taux de couverture global doit atteindre **au minimum 80 %**.